Graphic Era deemed to be University DEHRADUN

# PROJECT AND TEAM INFORMATION

## Project Title

| |
|---|
| Satellite Health Monitoring and Collision Avoidance |

| | |
|---|---|
| *Team Name:*<br>*Team #* | *Sentra*<br>*DSCPP-III-2026-T004* |
| **Team member 1 (Team Lead)**<br>*(Last Name, name: student ID: email, picture):* | *Chhavit – 2510011425*<br>*2510011425@geu.ac.in*<br>  |
| **Team member 2**<br>*(Last Name, name: student ID: email, picture):* | *Jain, Harsh – 2510370400*<br>*2510370400@geu.ac.in*<br>  |
| **Team member 3**<br>*(Last Name, name: student ID: email, picture):* | *Kumar, Satyam – 2510380092*<br>*2510380092@geu.ac.in*<br>  |
| **Team member 4**<br>*(Last Name, name: student ID: email, picture):* | *Ojha, Sudhanshu – 2510011648*<br>*2510011648@geu.ac.in*<br>  |

# PHASE-III PROJECT PROGRESS DESCRIPTION

## Project Abstract

The "Satellite Health Monitoring and Collision Avoidance" project is a simulation that addresses the growing problem of space debris in Earth's orbit. It models autonomous satellites that monitor their own health, repair software faults and perform evasive manoeuvres to avoid collisions without waiting for commands from the ground.

The core is a C++ program built on the Object-Oriented Programming (OOP) and Data Structures concepts of our syllabus. An AVL tree maps space into 3D sectors so each satellite searches only the 27 sectors around it for debris; a Min-Heap passes the nearest threat to the flight computer first; and a Circular Queue keeps each satellite's battery history. OOP classes model communication and imaging satellites, fuel use and gravity for circular orbits, and a Chaos Engine injects hardware faults, battery drains and new debris to test the system. Every position is written to `telemetry.csv` and every event to `blackbox.txt`.

In Phase III we built an interactive HTML, CSS and JavaScript website. It runs a JavaScript copy of our C++ simulation logic in the browser, on real satellite and debris data from CelesTrak or on satellites the user enters, and shows every alert, evasive burn, red zone and repair in 3D. Replaying files written by the C++ program is switched off until the C++ supports real-data runs.

## Updated Project Approach and Architecture

**C++ backend:** Modular headers (`Satellites_OOP.h`, `FlightComputer.h`, `Chaos_Engine.h`, `AVL_Tree.h`, `Queues_DSA.h`) run a fixed tick loop: chaos events, gravity, AVL tree rebuilt from sector keys, health checks, telemetry output, a 27-sector threat scan, and Min-Heap decisions (warning, collision alert, evasive burn or repair). Output: `telemetry.csv` and `blackbox.txt`.

**Website:** Plain HTML, CSS and JavaScript with no frameworks or libraries, delivered as a page (`index.html`, `style.css`) with seven script files in `js/` (core, data, simulator, render, playback, panels, ui), and as one offline file (`orbital.html`). A browser simulator, a JavaScript port of `orbital.cpp`, writes the same file formats in memory, which the website parses, indexes and draws. CelesTrak orbital elements are converted to starting positions with Kepler's equation and J2 drift, an approximation of SGP4. A Canvas 2D renderer projects 3D positions onto the screen, blends between written ticks, and draws the Earth, orbit paths, risk lines and red zones, with panels for satellites, events, a timeline, raw data and a Burn focus view.

**Additions found necessary on real data:** Closest-approach checking between ticks, joining docked station modules into one spacecraft, and a run-size check. No server is used; files are read locally. AI assistance was used in building the website.

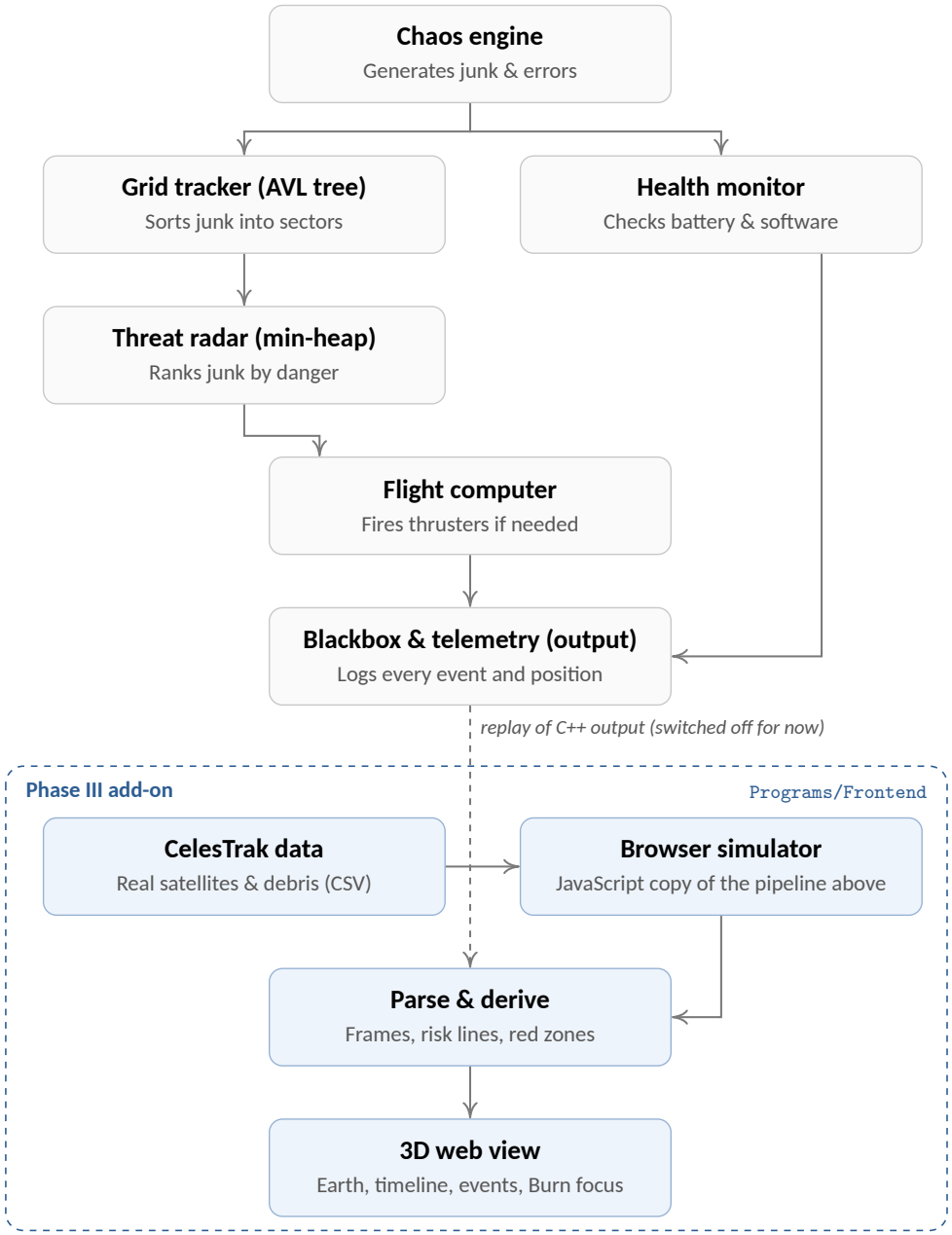



**Fig. Project Architecture**

## Tasks Completed

| Task Completed | Team Member |
|---|---|
| Designed the website: start screen, New run form, "How Orbital works" guide and footer. | All Team Members |
| Browser simulator (JavaScript copy of `orbital.cpp`), CelesTrak CSV import and parsers for the telemetry and event-log formats. | All Team Members |
| Canvas 2D 3D projection, Earth, orbit paths and camera views (isometric, top-down, follow). | Chhavit & Satyam Kumar |
| Interactive panels, playback timeline, raw data table and the Burn focus view. | Harsh Jain & Sudhanshu Ojha |

## Challenges/Roadblocks

**1. Fast debris missed between ticks:** Real debris crosses a satellite at about 10 km/s, so checking distances once per second missed most close passes. We now compute each pair's closest approach during the next tick ($t^* = -\mathbf{r} \cdot \mathbf{v}/|\mathbf{v}|^2$), which raised debris collision alerts from 1 to 3 on the same run. We plan to port this to the C++.

**2. Surprises in real data:** CelesTrak lists docked station modules (for example POISK on the ISS) with the station's own orbit. Simulated separately they sat at distance 0 and flooded one run with 131,443 false alerts. Objects within 1 km and 1 m/s of each other are now joined into one spacecraft.

**3. Huge output:** Writing every position every tick produced a 3.9 GB telemetry file. The website writes every $N$ ticks and estimates a run's size first, offering a one-click fix when it is too big.

**4. Speed:** With 4,000+ objects the close-up Burn focus view dropped to 2.5 frames per second. Drawing only the visible part of each orbit path, caching paths, limiting Earth image detail, and using solid panels instead of blurred ones on big runs raised it to about 38.

**5. Bugs found in our C++:** The sector key formula clashes once a sector index reaches 500, satellites burn fuel on every tick a threat lasts, repaired satellites freeze in place, and thresholds are hard-coded. Fixes are in progress.

## Project Outcome/Deliverables

The deliverables are an autonomous C++ simulation and an interactive website. The C++ program uses an AVL tree, Min-Heap, Circular Queue, linear search, inheritance, virtual functions and operator overloading, and writes `telemetry.csv` and `blackbox.txt`. The website (`Programs/Frontend`: `index.html`, `style.css` and seven script files in `js/`, or the single file `orbital.html`) simulates and shows runs in 3D, from real CelesTrak satellite and debris data or from satellites the user enters, with an event feed, timeline, raw data, risk lines, red zones and a Burn focus view. Supporting deliverables: CelesTrak download scripts for Windows and macOS/Linux, the in-app "How Orbital works" guide, and the project licence.

# Progress Overview

**Completed:** The C++ backend (AVL tree, Min-Heap, flight computer, chaos engine, file output) from earlier phases, and our main Phase III objective, the visual website, which is finished ahead of schedule and extended to real CelesTrak data with closest-approach checking.

**In progress:** The C++ has no real-data (kilometre) mode yet, does not write `run_info.txt`, and still has the bugs listed under Challenges. Because of this, replaying C++ output in the website is switched off for now, so the website currently shows its own simulation of our logic rather than C++ results.

**Pending:** Measuring the simulation's accuracy against SGP4, and comparing the AVL tree's speed with a linear scan.

# Codebase Information

Repository Link: `https://github.com/Chhavit09/SHMCA`
Branch: `main`

Phase III commits (website in `Programs/Frontend`, merged through a reviewed pull request):
- `2cd7b17`: Core state, parsers, browser simulator and CelesTrak import (harshj1214-hj)
- `c5b8648`: Frontend: page structure, start screen, New run form, guide and footer (Chhavit09)
- `6309fc3`: Frontend: 3D projection, Earth, orbit paths and camera views (kr-Satyam09, harshj1214-hj)
- `9d4cd9e`: Frontend: playback timeline, speeds and Burn focus view (harshj1214-hj, sudhanshuOjha07)
- `c44cc37`: Frontend: panels, event feed and raw data table (sudhanshuOjha07, harshj1214-hj)
- `42fb8db`: Merge pull request #2 from Chhavit09/phase3-website (Chhavit09)
- `6cdea6a`: Rename frontend folder to Frontend (kr-Satyam09)

Earlier phases: 60 commits on `main` for the C++ modules, by all four members (see the Phase II report).

# Testing and Validation Status

| **Test Type** | **Status (Pass/Fail)** | **Notes** |
|---|---|---|
| Closest-approach check | Pass | Four geometric test cases all give the expected distance. |
| Docked-module joining | Pass | ISS with POISK joined: 131,443 false alerts reduced to 0. |
| Log event synchronisation | Pass | Every event belongs to a known satellite and lines up with the timeline. |
| Large input and speed | Pass | 180 satellites and 4,070 debris; Burn focus 2.5 $\rightarrow$ 38 frames/s. |
| C++ code review | Fail | Sector key clash, fuel per tick, frozen repair; fixes in progress. |

# Deliverables Progress

| **Deliverable** | **Status** |
|---|---|
| Modular C++ Backend (OOP & DSA) | Completed |
| Flight Computer and Chaos Engine | Completed |
| Telemetry and Blackbox File Output | Completed |
| HTML/CSS/JS Website with real CelesTrak data | Completed |
| Download scripts, in-app guide and licence | Completed |
| C++ real-data mode and bug fixes | In Progress |
| Accuracy check against SGP4 | Pending |